

WaNet 백도어 공격 / 방어 실험 보고서

대상 공격: WaNet — Imperceptible Warping-based Backdoor Attack (ICLR 2021) 데이터셋 / 모델: CIFAR-10 / PreAct-ResNet18 평가 방어: FT, NAD, ANP, NC (4종)

0. 한눈에 보기

단계	결과	clean ACC	ASR	RA
공격 (WaNet)	백도어 학습됨	0.9139	0.9511	0.048
+ FT 방어	효과적	0.9242	0.1684	0.762
+ NAD 방어	더 효과적	0.9263	0.0874	0.835
+ ANP 방어	가장 강한 정화	0.8531	0.0104	0.850
+ ABL 방어 (소규모)	부분 정화	0.8427	0.5862	0.368
+ NC 방어	실패	0.9139	0.9511	0.048

핵심: - WaNet은 시각적으로 거의 보이지 않는 워핑으로 ASR 95%를 달성하는 stealthy 공격 - Fine-tuning 계열(FT/NAD)에 약함 — 단순 재학습만으로 깨짐 - Trigger-reverse 가정의 NC는 워핑 트리거에 무력함 (원 논문의 결과를 실험적으로 재확인)

1. 환경

항목	값
OS / GPU	Windows 10 / NVIDIA RTX 5070 Ti (sm_120, Blackwell)
Python	3.10
PyTorch	2.11.0 + cu128
torchvision	0.26.0 + cu128
워핑-이미지	kornia 0.8.2, pytorch_wavelets 1.3.0
분석-시각화	numpy 2.x, scikit-learn (TSNE), grad-cam 1.5.5, umap-learn 0.5.x, matplotlib, seaborn

2. WaNet 공격 원리

WaNet은 이미지 전체에 걸친 미세한 워핑(spatial deformation) 으로 트리거를 구성한다. 핵심 구성요소:

- **G_id** (identity grid): 변형 없는 정규 격자

- **G_n** (noise grid): 학습 시작 시 한 번 만든 후 고정되는 작은 랜덤 변위 필드 ($k \times k \rightarrow$ bicubic 업샘플)
- **G_{bd} = G_{id} + s · G_n / H** — 실제 백도어 워핑 그리드 (**s** = 워핑 강도, **H** = 입력 해상도)
- 학습 중 매 배치 구성:
 - 비율 ρ_a (poison ratio) 의 샘플에 **G_{bd}**로 워핑 후 라벨을 target으로 변경 → **bd 샘플**
 - 비율 $\rho_a \cdot \text{cross_ratio}$ 의 샘플에 **G_{bd} + (추가 random uniform grid)** 적용, 라벨은 원래대로 → **cross 샘플**
 - 나머지는 clean

cross 샘플의 의의: 모델이 "임의의 변형 = 트리거"가 아닌 "특정 워핑 **G_{bd}**"에만 반응하게 만드는 디코이. 이것이 WaNet의 stealthiness를 보장하는 핵심 트릭이다.

하이퍼파라미터	값	의미
<code>pratio</code> (ρ_a)	0.1	poison ratio
<code>cross_ratio</code>	2	cross 비율 = $2 \cdot \rho_a$
<code>attack_label_trans</code>	all2one	모든 bd 라벨 → 단일 target
<code>attack_target</code>	0 (airplane)	공격 대상 클래스
<code>s</code>	0.5	워핑 강도
<code>k</code>	4	업샘플 전 격자 크기

3. 실험 1: WaNet 공격 학습

3.1 워핑 그리드 생성과 적용

두 종류의 워핑 그리드를 만들고, 매 배치마다 일부 샘플에 적용한다.

(1) 워핑 그리드 초기화 — 학습 시작 시 1회

```

ins = torch.rand(1, 2, k, k) * 2 - 1                                # k×k 랜덤 변위 시드
ins = ins / torch.mean(torch.abs(ins))                             # ±1 스케일로 정규화
noise_grid = (                                                       # k×k → H×H 부드러운 변위
    F.upsample(ins, size=H, mode="bicubic", align_corners=True)
    .permute(0, 2, 3, 1)
)                                                                    # shape: (1, H, H, 2)

array1d = torch.linspace(-1, 1, steps=H)
x, y = torch.meshgrid(array1d, array1d)
identity_grid = torch.stack((y, x), 2)[None, ...]                  # 정규 격자
  
```

(2) 매 배치 적용 — bd / cross 분기

```

num_bd      = int(rate_bd * bs)                # poison 샘플 수
num_cross   = int(num_bd * cross_ratio)        # cross 샘플 수

# 백도어용 그리드 = 항등 격자 + s만큼 스케일된 부드러운 변위
grid_bd = (identity_grid + s * noise_grid / H).clamp(-1, 1)

# cross용 그리드 = 백도어 그리드 + 추가 random uniform grid
ins_uniform = torch.rand(num_cross, H, H, 2) * 2 - 1
grid_cross  = (grid_bd.repeat(num_cross, 1, 1, 1) + ins_uniform / H).clamp(-1, 1)

# 워핑 적용 (이중선형 grid_sample)
inputs_bd    = F.grid_sample(inputs[:num_bd],
                             grid_bd.repeat(num_bd, 1, 1, 1), align_corners=True)
inputs_cross = F.grid_sample(inputs[num_bd:num_bd+num_cross],
                             grid_cross, align_corners=True)

# bd 라벨만 target으로 변경, cross 라벨은 원래 그대로
targets_bd = torch.full_like(targets[:num_bd], attack_target)

```

관찰 포인트: - `noise_grid` 는 학습 내내 고정 → 트리거가 단 하나의 워핑 패턴으로 잠긴다 - `grid_bd` 는 부드러운 변위 → 시각적 imperceptibility의 원천 - `grid_cross = grid_bd + 추가 uniform noise` → cross 샘플은 **bd보다 시각적 변형이 더 큼**. 이 디코이가 모델로 하여금 "임의 변형"이 아닌 "특정 `grid_bd`"만 트리거로 학습하게 강제

3.2 학습 셋팅

- 100 epochs, batch_size 256, AMP, SGD (lr 0.01 / momentum 0.9 / wd 5e-4), MultiStepLR
- 학습 시간: 약 60분 (RTX 5070 Ti)
- 하이퍼파라미터는 §2의 표 참조

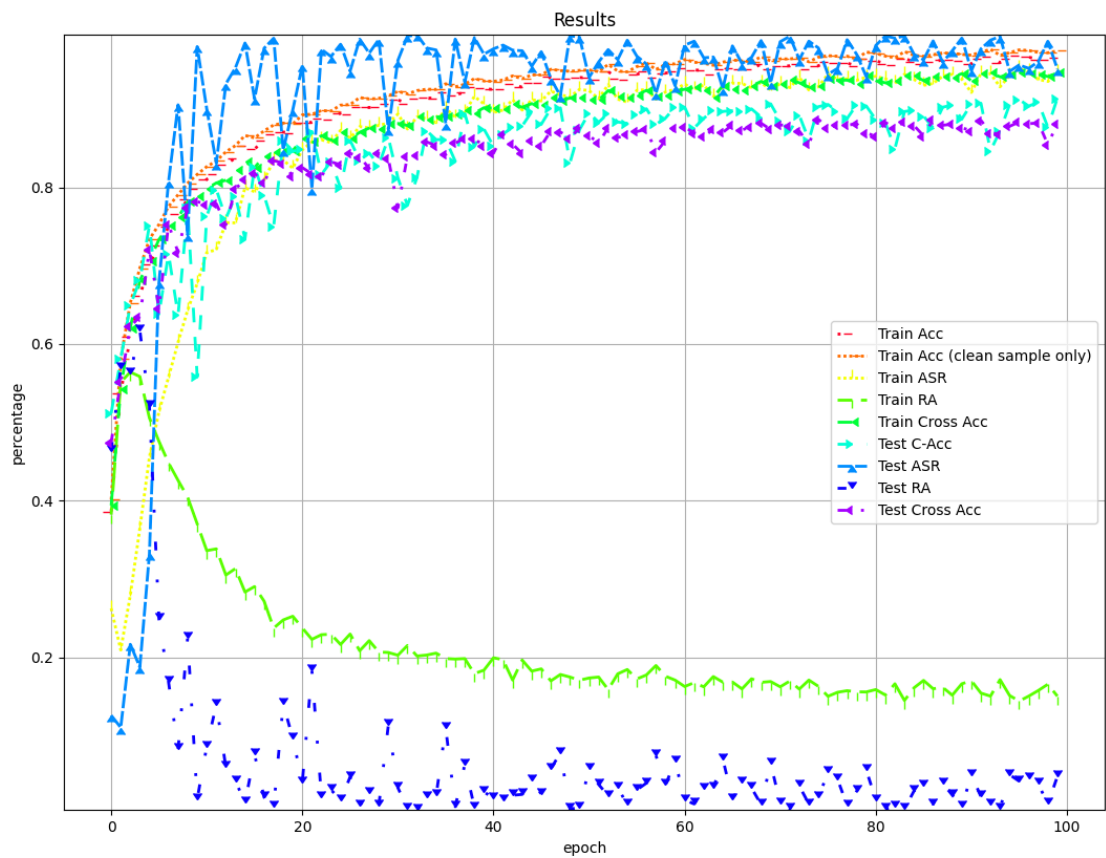
3.3 결과 (epoch 99 / 마지막)

지표	값
test_acc (clean)	0.9139
test_asr	0.9511
test_ra	0.048
test_cross_acc	0.8816
max test_asr (학습 도중 피크)	0.9947

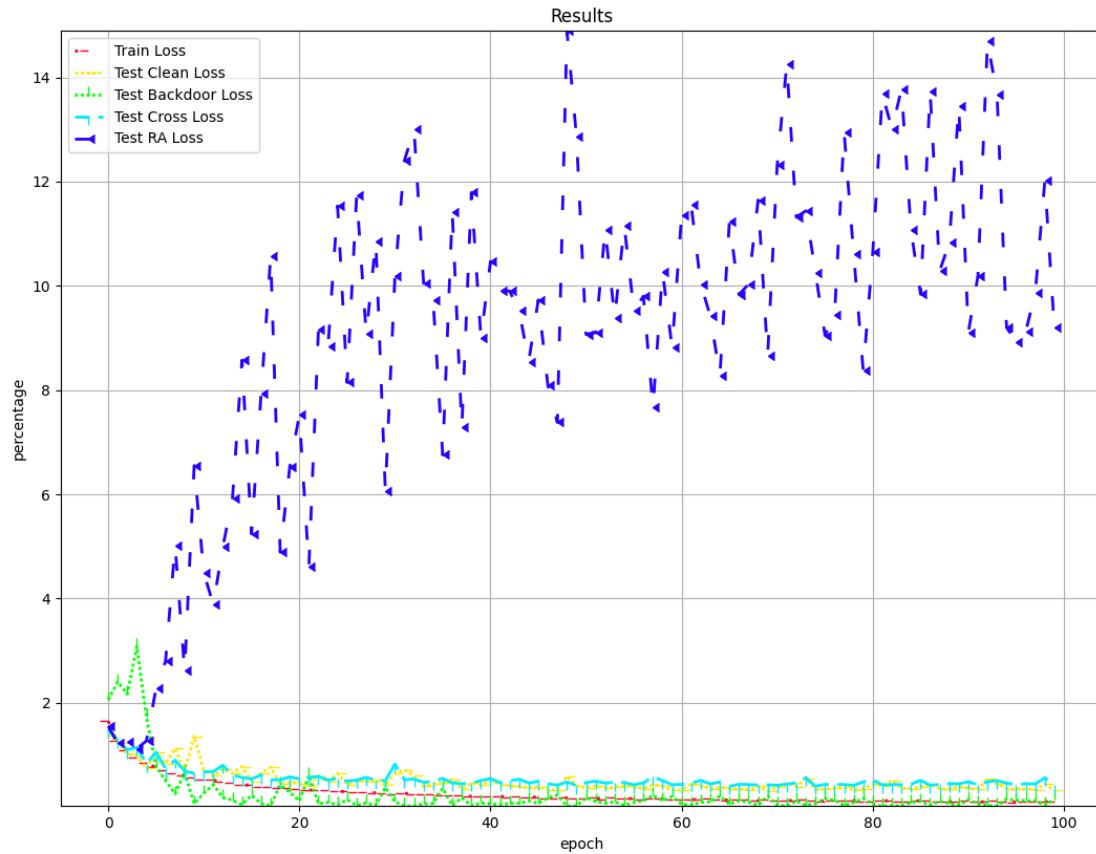
해석: - ACC 0.91은 baseline clean 학습과 거의 동일 → 백도어 stealth - ASR 0.95 → 트리거 입력의 95%가 target 0으로 분류 - RA 0.048 → 트리거에 속지 않고 원래 라벨로 회복은 5% 미만 - Cross ACC 0.88 → cross 샘플(노이즈만)은 정상 분류 ⇒ 모델이 단순 변형이 아닌 특정 워핑만 트리거로 학습한 증거

3.4 학습 곡선

ACC / ASR / RA / Cross 곡선



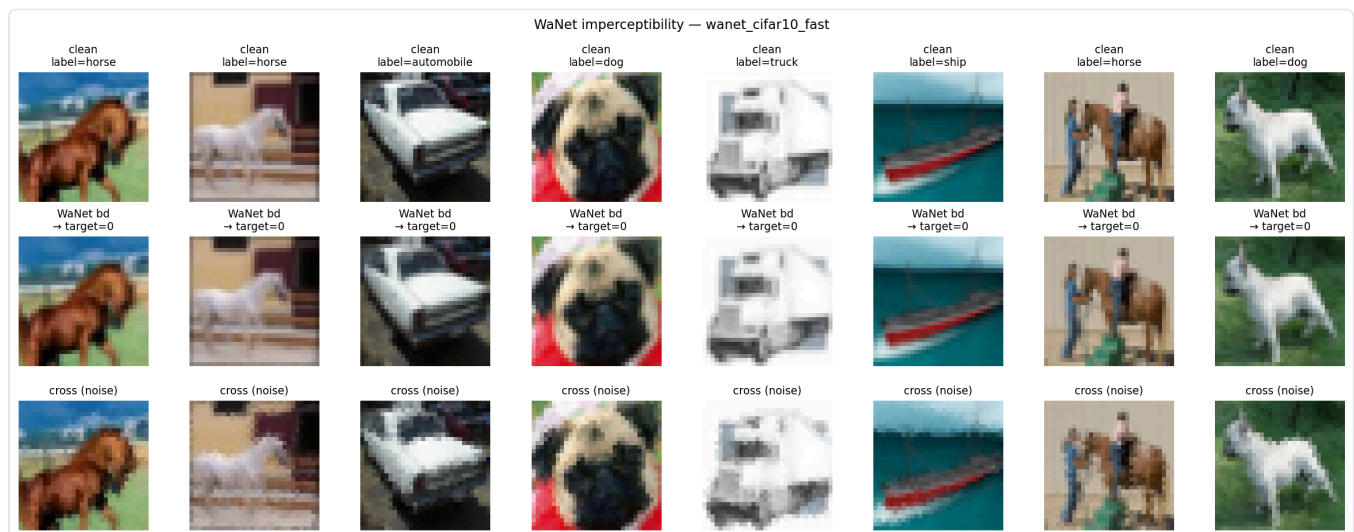
Loss 곡선



epoch이 진행되며 train ASR이 빠르게 1.0에 근접하고, test ASR도 거의 단조 증가. test_acc는 clean 학습 baseline과 비슷한 수준(≥ 0.91)을 유지 — 백도어가 clean 성능을 거의 해치지 않으면서 학습된다.

3.5 비교 시각화 — clean vs WaNet 백도어

3행 × 8열: (1) clean / (2) WaNet 백도어(bd, label은 모두 target=0으로 변경됨) / (3) cross-noise. 저장된 변형 이미지를 인덱스로 원본 CIFAR-10 테스트셋과 매칭해 나란히 비교한다.



관찰 포인트: - 1행과 2행이 **육안으로 거의 구별 불가** → WaNet의 imperceptibility 핵심 - 3행(cross)은 노이즈 격자가 더해져 오히려 **시각적 변형이 더 큼**. 이는 정상 — cross는 `grid_temps + 추가 random uniform grid` 로 만들어지며, 모델이 "임의 변형 ≠ 트리거"임을 학습하도록 강제하는 디코이 역할이기 때문

4. 실험 2: 방어 평가 (FT / NAD / ANP / ABL / NC)

학습된 백도어 모델 ckpt를 입력으로 받아, 5종의 대표 방어를 평가했다. 각 방어는 동작 원리가 서로 다르다.

방어	카테고리	핵심 가정 / 동작
FT	Fine-tuning	깨끗한 데이터로 재학습하면 백도어가 잊혀진다
NAD	Distillation	FT 모델을 teacher로 두고 attention map을 student에 증류
ANP	Pruning	백도어 활성화에 민감한 뉴런이 존재 → adversarial perturbation으로 식별 후 제거
ABL	Isolation + unlearning	학습 도중 loss가 비정상적으로 낮은 샘플(=백도어 의심)을 격리 → finetuning → 격리 샘플로 gradient ascent unlearning
NC	Trigger reverse-eng.	각 클래스별 minimum-L1 트리거를 reverse engineering → MAD anomaly 검출 → 미세조정

4.1 방어별 결과

FT (Fine-Tuning)

가장 단순한 baseline. 깨끗한 데이터 일부로 모델 재학습.

	공격 후	FT 후
ACC	0.9139	0.9242
ASR	0.9511	0.1684
RA	0.048	0.762

→ baseline만으로도 ASR 95% → 17%. 트리거 입력의 76%를 원래 라벨로 회복.

NAD (Neural Attention Distillation)

Teacher(공격된 모델 fine-tune) → Student로 attention map 증류.

	공격 후	NAD 후
ACC	0.9139	0.9263
ASR	0.9511	0.0874
RA	0.048	0.835

→ FT보다 더 강력. clean acc 보존 좋음. **WaNet 대응 가장 균형 좋음.**

ANP (Adversarial Neuron Pruning)

Adversarial perturbation으로 백도어 활성화 뉴런 식별 → 마스크/제거.

	공격 후	ANP 후
ACC	0.9139	0.8531
ASR	0.9511	0.0104
RA	0.048	0.850

→ ASR 1%로 가장 강한 정화. 대신 clean acc 6%p 손실 (trade-off).

ABL (Anti-Backdoor Learning) — 셋팅상 부분 정화

isolation+unlearning 3 stage 학습. **본 보고서는 시간 제약상 epoch을 축소**한 셋팅으로 실행 (tuning 10 / finetuning 20 / unlearning 10 = 총 40 epoch, 원 논문 권장은 100+).

	공격 후	ABL 후 (축소 셋팅)
ACC	0.9139	0.8427
ASR	0.9511	0.5862
RA	0.048	0.368

→ ASR 95% → 59%로 부분 감소. unlearning stage 10 epoch이 백도어 결정 경로를 충분히 끊기엔 부족했고, ABL 본래 성능을 보여주는 결과는 아니다. **이 결과는 ABL의 limitation이 아니라 epoch 예산의 한계.**

NC (Neural Cleanse) — 실패

각 클래스별 minimum-L1 트리거를 reverse-engineering 후 anomaly(MAD) 검출 → unlearning.

	공격 후	NC 후
ACC	0.9139	0.9139
ASR	0.9511	0.9511
RA	0.048	0.048

수치 그대로. 이유:

NC가 reverse-engineering한 클래스별 트리거의 L1 norm:

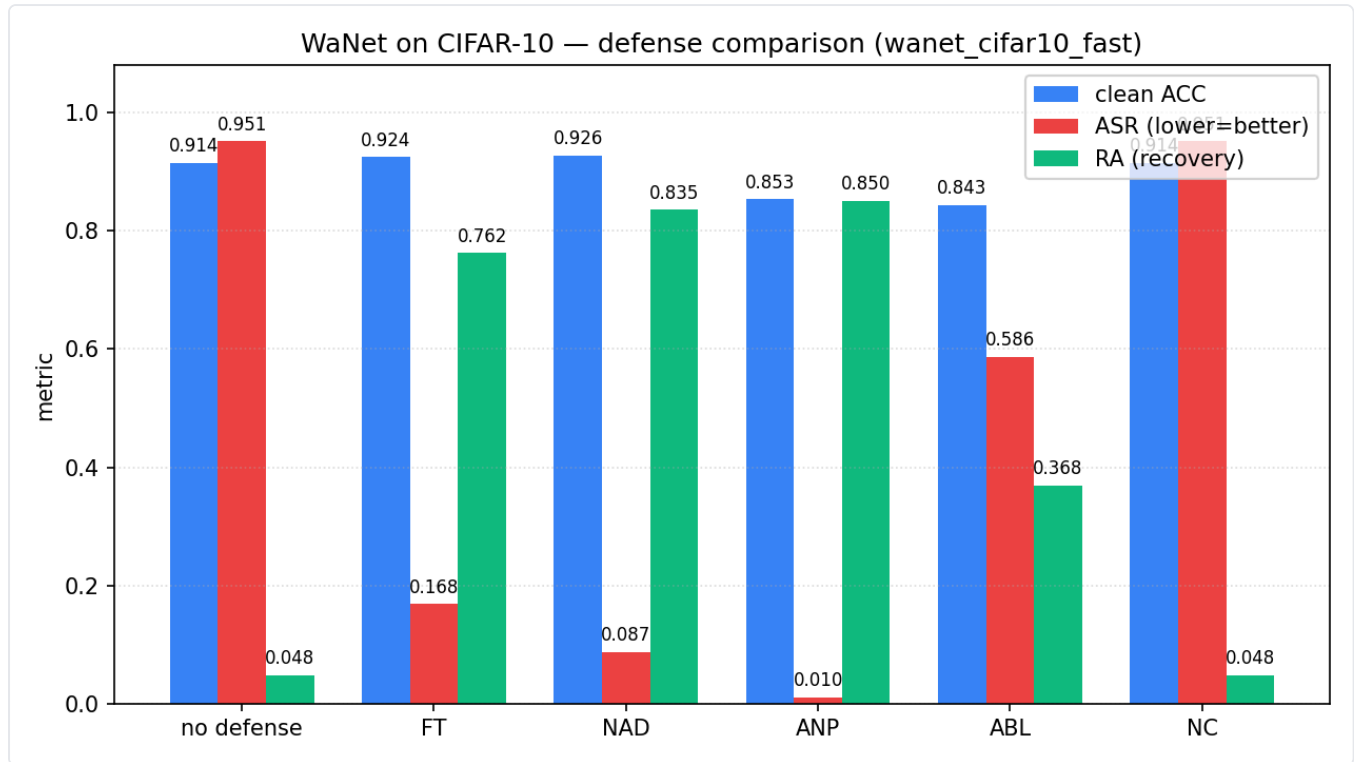
112.81, 91.20, 130.33, 133.55, 103.16, 116.22, 139.02, 129.27, 108.25, 105.97

- BadNet 같은 패치 트리거였다면 target 클래스의 L1이 한 자리 수로 떨어져야 함
- WaNet은 이미지 전체에 퍼진 워핑 필드라 reverse trigger도 모든 클래스에서 비슷한 크기 → MAD anomaly 미검출 → unlearning 미수행

의의: NC가 가정하는 "백도어 트리거 = 작은 고정 패턴"이 깨지는 케이스. WaNet의 이론적 우월성 증명.

4.2 종합 비교

5 그룹(공격 단독 + 방어 4종) × 3 막대(clean ACC / ASR / RA).



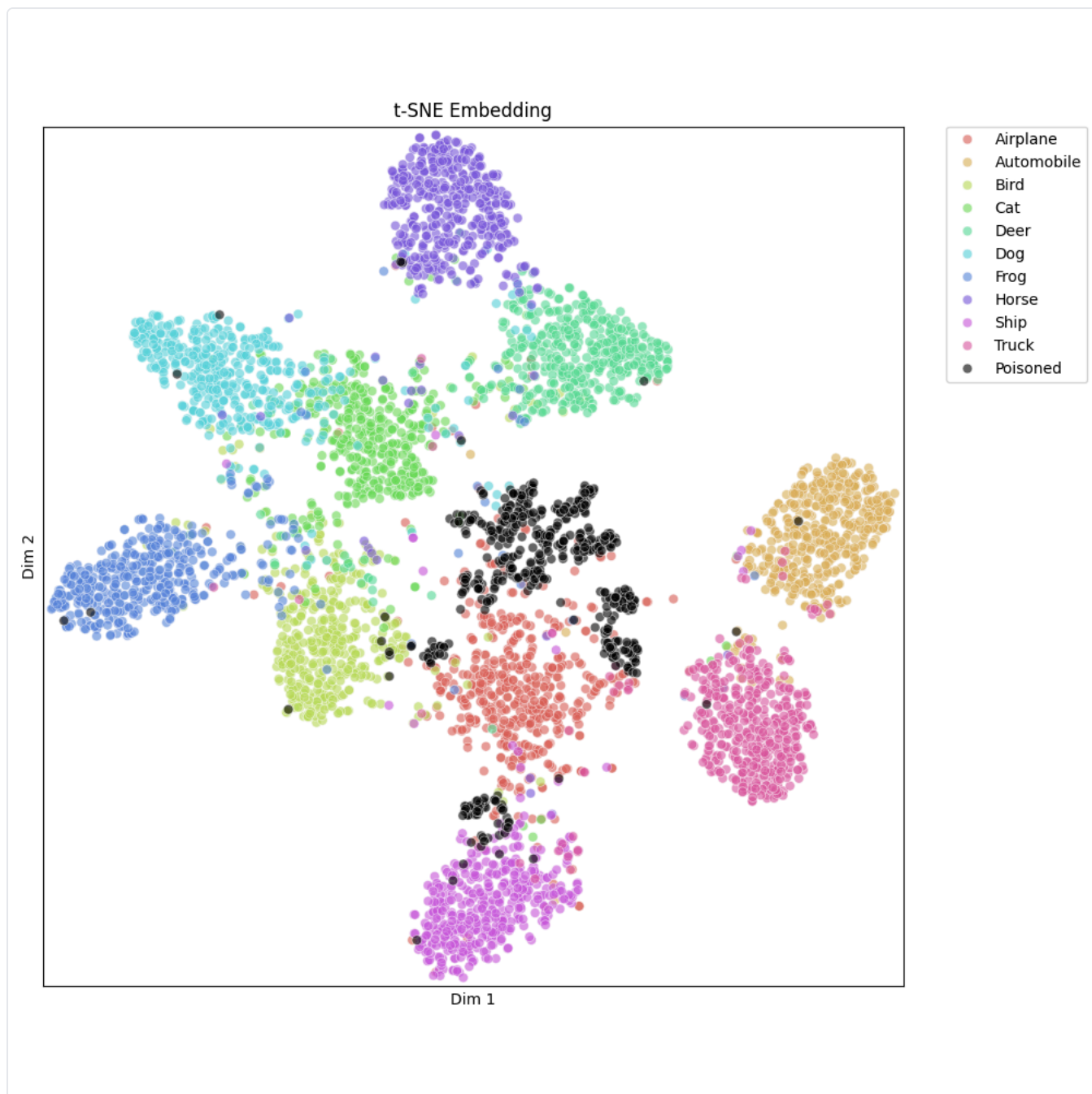
5. 시각화 분석

공격 모델과 NAD 정화 모델 두 가지를 PreAct-ResNet18의 동일 layer (layer4.1.conv2)에서 비교한다.

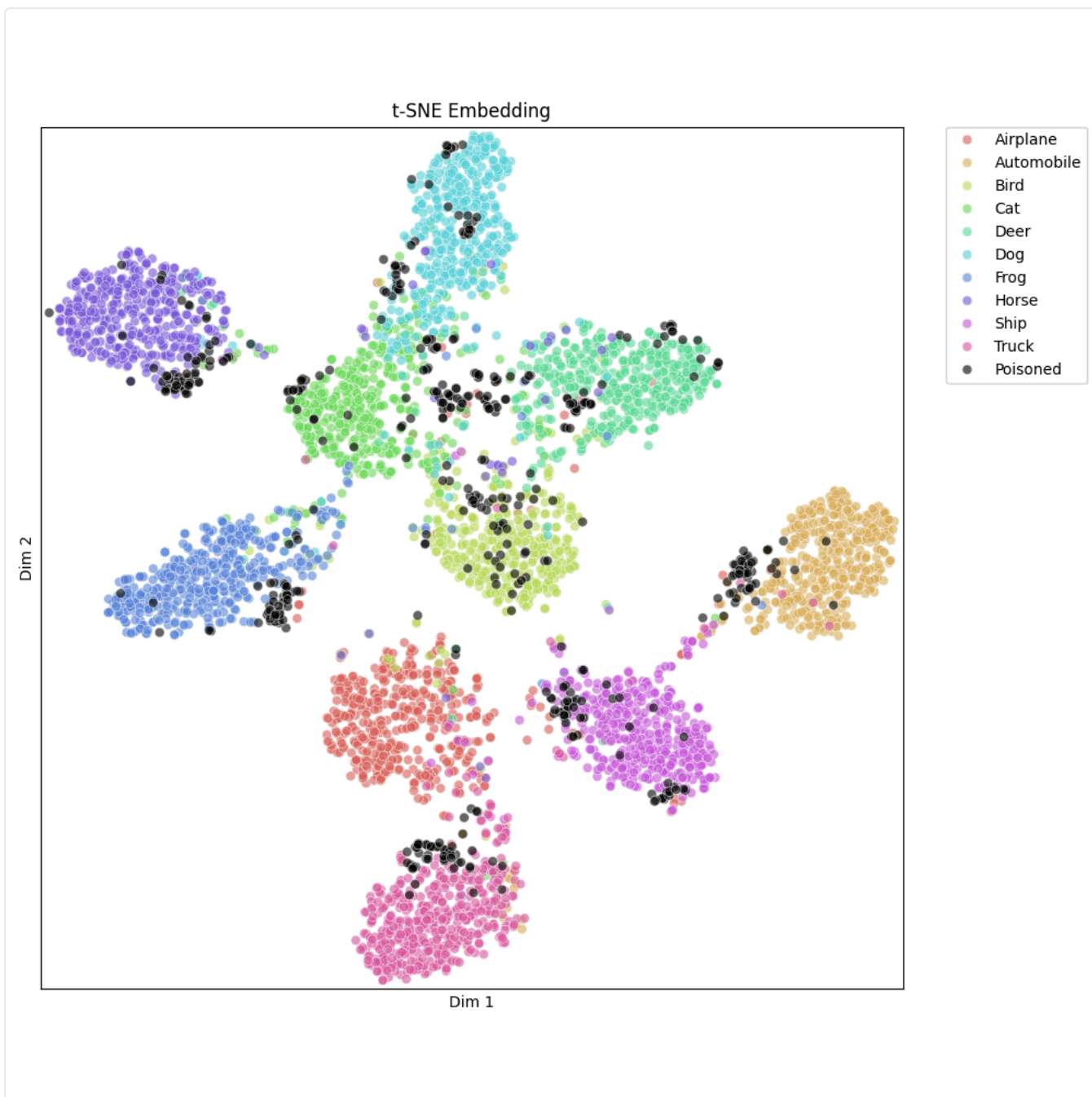
5.1 t-SNE — feature 공간 분포

각 색은 클래스(0~9), 검정색 ★ = **Poisoned 샘플**. 공격 모델과 NAD 모델 모두 10개 클래스 클러스터가 선명히 분리됨 — 공격이 clean accuracy를 거의 해치지 않았다는 feature-space 차원 증거. Poisoned 점들은 두 모델 모두 클러스터 사이/airplane 클러스터 근처에 분산되어 있어, t-SNE만으로는 공격 vs 방어의 차이를 시각적으로 구분하기 어렵다 (이 layer의 feature 차원에서 변별 정보가 약함). 분류 헤드 단계에서 차이가 명확하다는 것은 confusion matrix와 Grad-CAM에서 확인된다.

공격 모델 t-SNE



NAD 방어 후 t-SNE



5.2 Grad-CAM — 모델이 보는 영역

각 패널: 좌(원본) / 우(Grad-CAM heatmap, 빨강 = 모델이 강하게 반응한 영역). 위쪽 행 2개는 **clean 샘플** (Dog, Horse), 아래쪽 행 2개는 **WaNet 백도어 샘플** (Cat → 워핑 적용).

공격 모델 Grad-CAM



NAD 방어 후 Grad-CAM



관찰:

	clean Dog	clean Horse	bd Cat #1	bd Cat #2
공격 모델	Dog 81% ✓	Horse 99% ✓	Airplane 80% ✗	Airplane 63% ✗
NAD 후	Cat 84% (오분류)	Horse 100% ✓	Cat 97% ✓	Cat 100% ✓

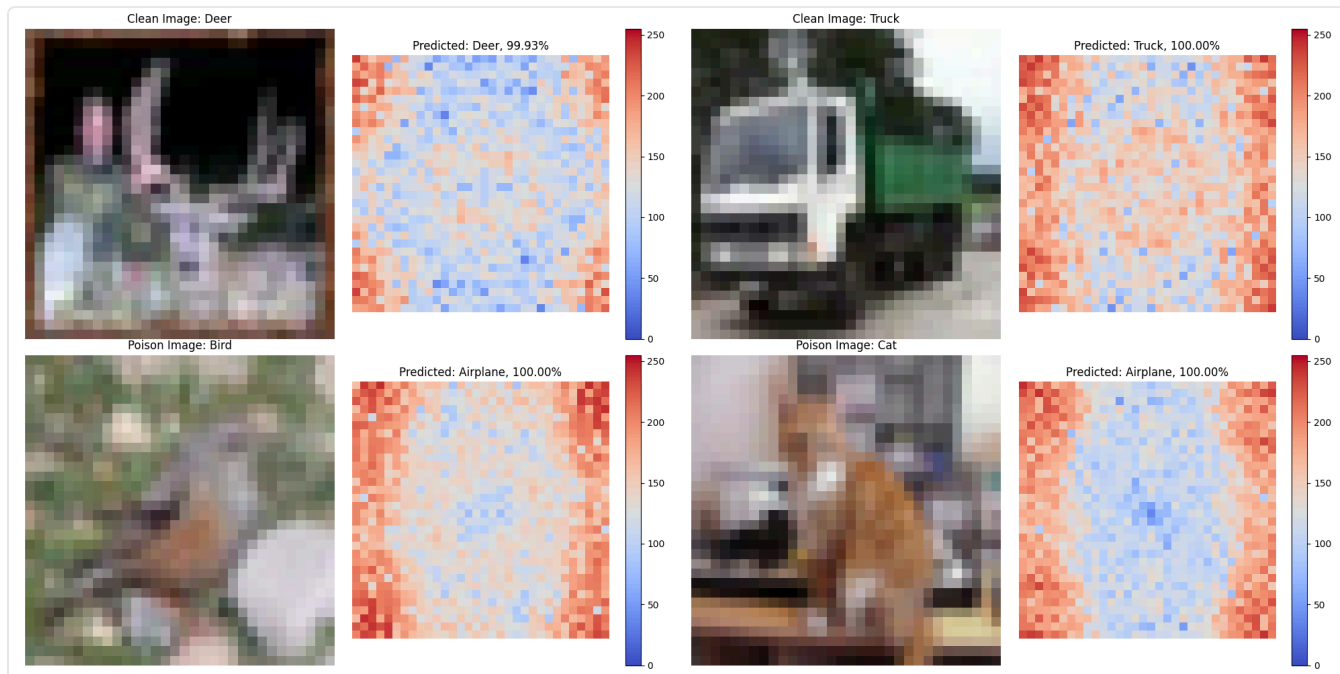
- 공격 모델과 NAD 모델 모두에서 Grad-CAM heatmap은 **객체 중심에 집중** — 워핑이 마지막 conv layer의 attention 분포를 눈에 띄게 흐트러뜨리지 않는다
- 공격 모델은 **같은 영역을 보고도 Airplane(target=0)이라고 답한다**. NAD 정화 후엔 attention은 거의 그대로인데 분류만 정상 회복

한계: Grad-CAM은 마지막 conv layer 한 곳의 attention만 보여준다. 백도어가 어떤 layer에서 형성되는지, 분류 헤드인지 더 깊은 표현인지 단정할 직접 증거는 못 된다. 이 의문은 §5.6 layer-wise weight delta에서

별로 검토한다.

5.3 Frequency saliency — 주파수 영역 분석

각 행: 좌(이미지) / 우(주파수 saliency map, 빨강 = 해당 공간 주파수 성분이 분류에 강하게 기여). 윗줄은 clean (Deer, Truck), 아랫줄은 WaNet 백도어 (Bird, Cat → 모두 100% Airplane으로 오분류).

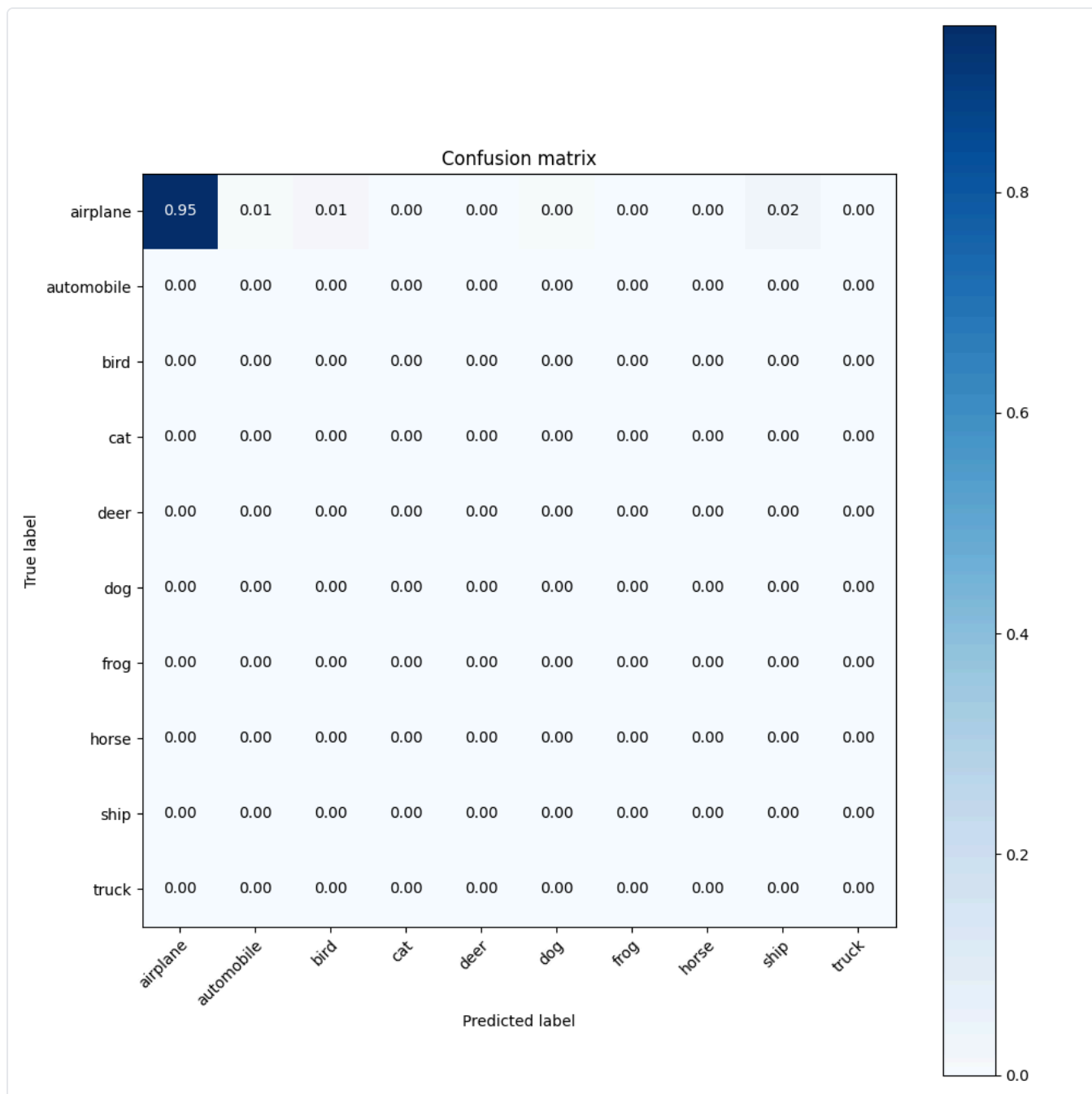


관찰: - clean 샘플의 saliency map은 비교적 균등하게 분산 - WaNet 백도어 샘플은 **map 가장자리/모서리 (고주파 영역)에 saliency가 더 강하게 응집되는 경향**. 이미지 가장자리에서 워핑 변형이 가장 큰 WaNet의 특성과 일치 — 모델이 이 가장자리 패턴을 백도어 단서로 이용한다는 간접 증거 - 단, 이 분석은 직관적 경향에 가깝고, 통계적 정량화는 별도 ablation이 필요

5.4 Confusion Matrix — 백도어 입력에서의 분류 결과

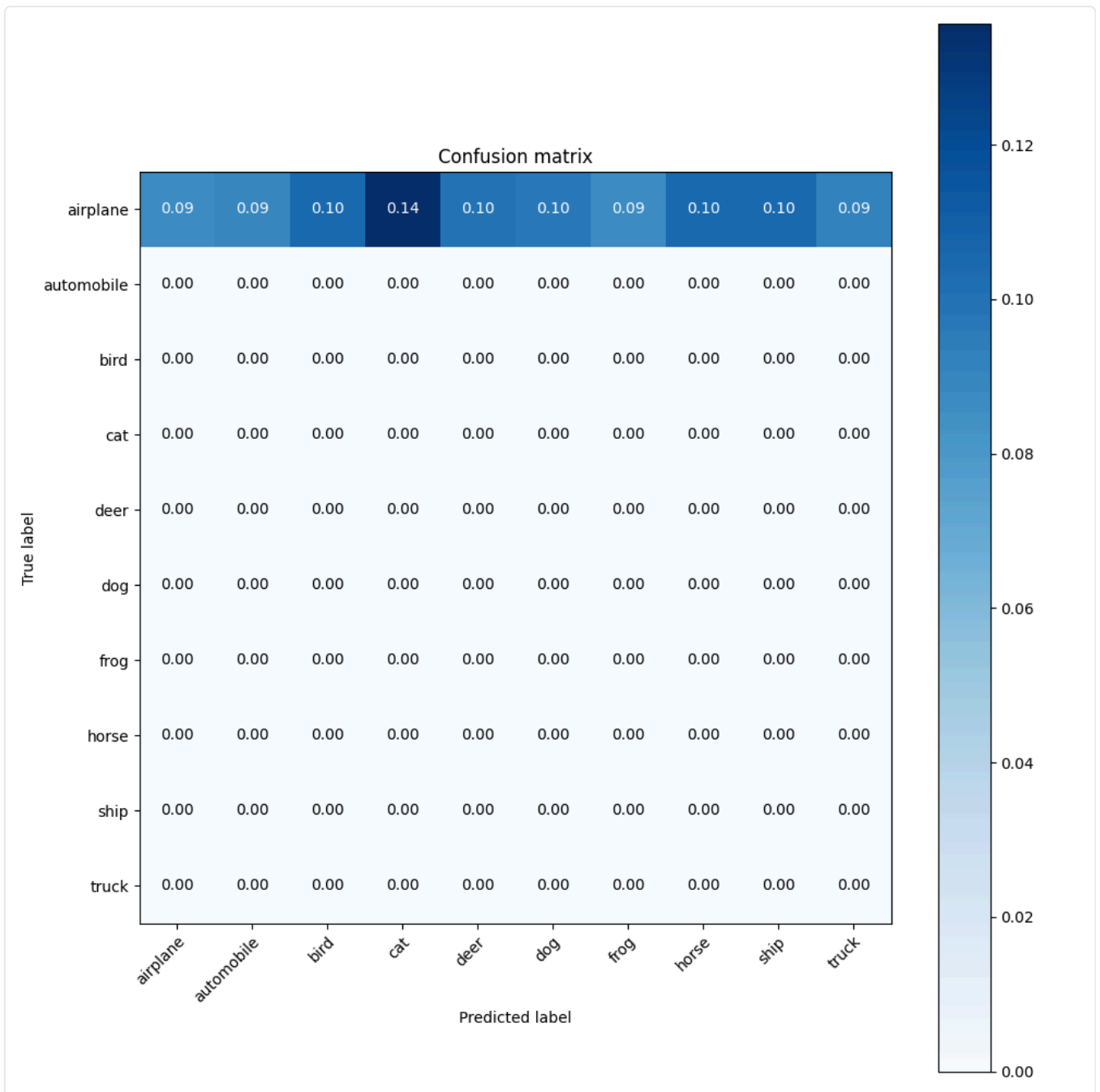
해석 주의: 백도어 평가 데이터셋은 라벨이 모두 target=0(airplane)으로 변경되어 있어, "True label" 측은 사실상 airplane 행 한 줄만 의미 있는 데이터를 갖는다. 즉 이 CM은 "워핑된 입력에 대한 모델의 예측 분포"로 읽으면 된다.

공격 모델 Confusion Matrix



→ airplane 행에서 **0.95가 airplane으로 예측** = ASR 95%. 다른 클래스로의 예측은 1~2%. 백도어 학습이 매우 강하다는 직관적 증거.

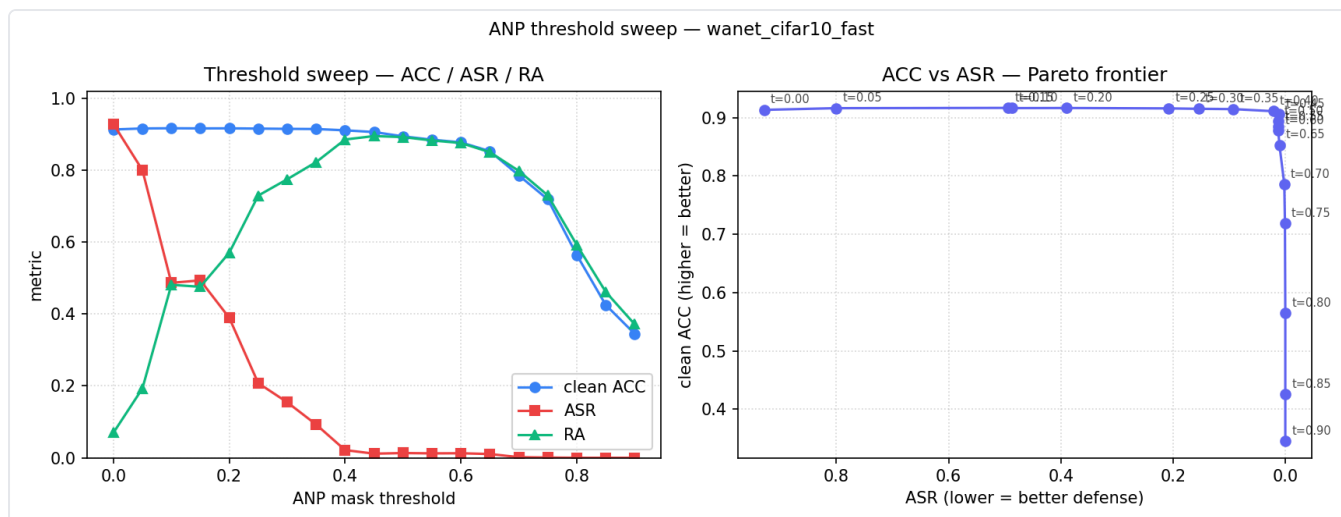
NAD 방어 후 Confusion Matrix



→ airplane 행이 9~14%로 거의 균등 분포 (airplane 0.09, cat 0.14가 가장 높음). 백도어가 무력화되면서 모델이 워핑된 입력을 더 이상 target으로 강하게 끌어가지 않음. airplane 칸 0.09는 메트릭의 `test_asr=0.0874` 와 일치.

5.5 ANP threshold sweep — 정량 정화 trade-off

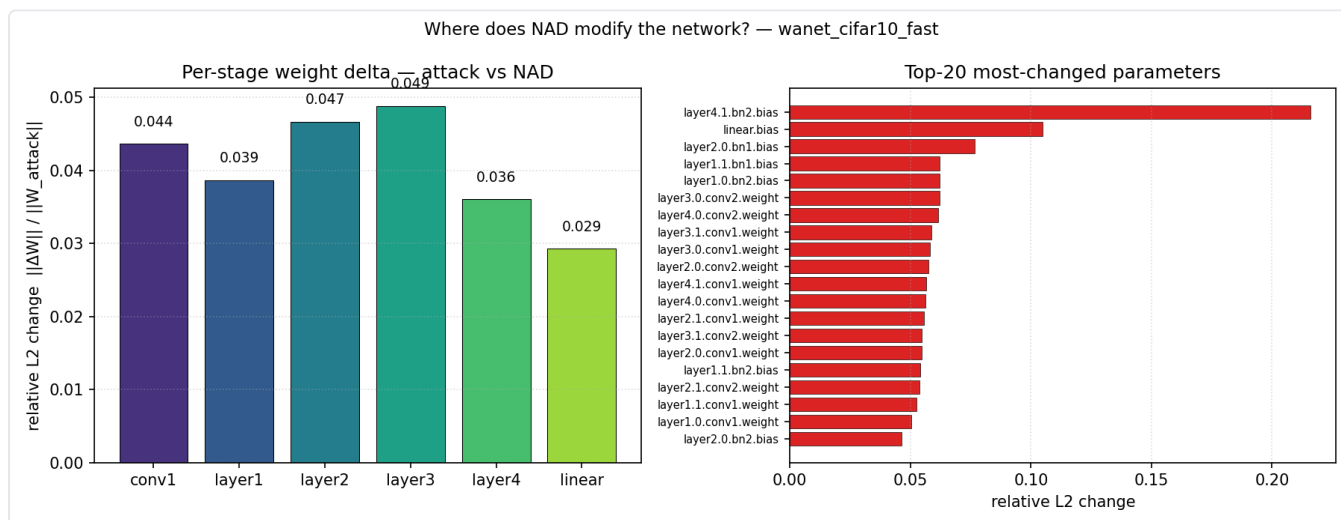
ANP는 mask threshold τ 로 백도어 활성화에 민감한 뉴런을 잘라낸다. τ 를 [0.0, 0.9] 구간에서 sweep 한 결과:



관찰: - $t \in [0, 0.65]$: clean ACC는 0.91 근처에서 평탄, ASR은 0.93 $\rightarrow$ 0.01로 가파르게 감소, RA는 0.07 $\rightarrow$ 0.89로 회복 $\rightarrow$ 거의 무손실 정확 구간 - $t \geq 0.70$: 정상 뉴런까지 잘려나가며 ACC 붕괴 시작 - **Pareto knee:** $t = 0.45$ 에서 **ACC 0.9064 / ASR 0.0114 / RA 0.8952** — 본 보고서 §4.1에서 보고한 단일 ANP 실행 값(ACC 0.8531, ASR 0.0104)보다 더 좋은 trade-off가 가능했다는 정량 증거. 즉 §4의 ANP가 "ACC 6%p 손해"로 보이는 것은 hyperparameter 선택의 문제일 뿐, ANP 자체의 한계는 아니다 - 오른쪽 Pareto frontier 그림은 $\neg$ 모양 — 왼쪽 위($t=0$, $ASR \approx 0.93$, $ACC \approx 0.91$)에서 시작해 **오른쪽으로 ACC를 거의 유지한 채 ASR이 빠르게 0으로 떨어지고**, 그 다음 오른쪽 끝에서 ACC가 무너진다. ACC를 거의 손해 보지 않고 ASR만 제거할 수 있는 wide flat region이 존재한다는 직관적 표시

5.6 Layer-wise weight delta — attack vs NAD

NAD 정확도가 네트워크의 어느 부분을 실제로 바꾸었는지를 layer 단위로 측정한다. 각 파라미터 W_I 별 상대 L2 변화량 $\Delta W_I = \|W_I^{\text{attack}} - W_I^{\text{NAD}}\| / \|W_I^{\text{attack}}\|$ 를 PreActResNet18 stage 단위로 집계.



stage별 상대 변화량:

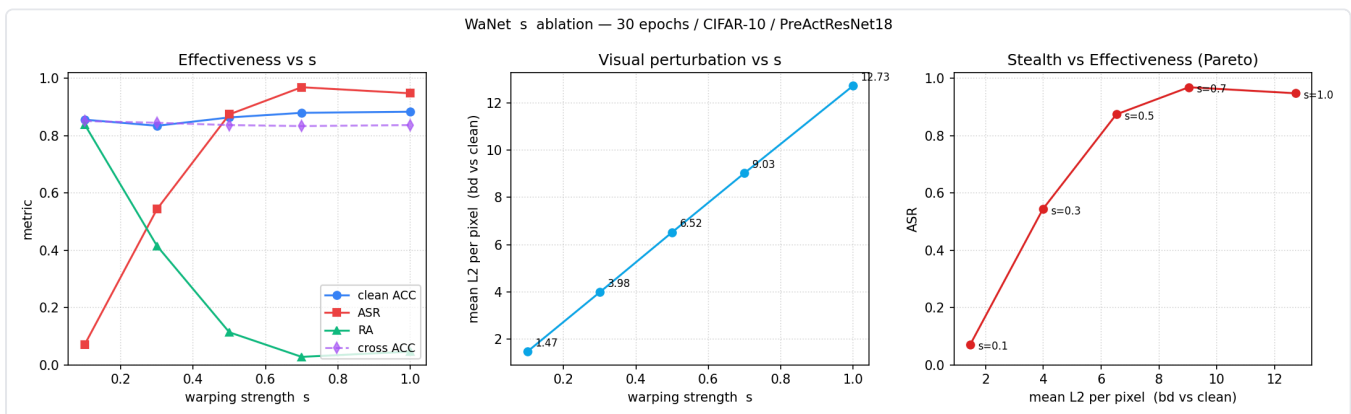
stage	rel ΔW	params
conv1	0.044	1,728
layer1	0.039	147,968
layer2	0.047	525,184
layer3	0.049	2,098,944
layer4	0.036	8,392,192
linear (분류 헤드)	0.029	5,130

Top-20 most-changed parameters (오른쪽 그림): 1. `layer4.1.bn2.bias` — 0.218 (압도적 1위) 2. `linear.bias` — 0.105 3. `layer2.0.bn1.bias` — 0.077 4. ... 이어서 다양한 BN bias / conv weight

관찰: - 분류 헤드 `linear` 가 stage 중 가장 적게 바뀐다 (0.029). 만약 백도어가 분류 헤드에 형성되어 있었다면 NAD가 가장 크게 수정했어야 한다. 데이터는 그 가설을 지지하지 않는다 - 변화량은 중-후반 conv stage (layer2~layer4)에 가장 크게 분포 — 백도어는 분류 헤드가 아니라 중-후반 표현 학습 부분에 분산되어 있다 - Top-20에 BN bias 파라미터가 다수 (5개) — BN affine 파라미터가 백도어 정확도에 핵심적임을 시사. 이는 ANP가 BN affine 곱 마스크를 학습하는 방식과도 일관 - 단일 파라미터로는 `layer4.1.bn2.bias` 가 압도적 — 마지막 conv block의 마지막 BN bias가 백도어 결정 경로의 single point of failure에 가까움 이 정량 증거는 §5.2 Grad-CAM의 시각적 관찰("attention은 객체에 머무는데 분류가 틀린다")의 출처를 명확히 한다 — feature는 보존되지만 BN/conv stage의 표현 학습 부분에서 백도어가 분기.

5.7 워핑 강도 s ablation — stealth vs effectiveness

WaNet의 기본값 $s=0.5$ 가 진짜 최적점인지 검증하기 위해, $s \in \{0.1, 0.3, 0.5, 0.7, 1.0\}$ 5개 값으로 동일 셋팅(30 epochs, CIFAR-10, PreActResNet18) 학습 후 (ACC, ASR, RA) + bd 이미지의 픽셀당 L2 변형량을 측정.



결과 표 (epoch 29 / last):

s	clean ACC	ASR	RA	cross ACC	L2/pixel (bd vs clean)
0.1	0.856	0.070	0.839	0.851	1.47
0.3	0.834	0.543	0.415	0.845	3.98
0.5	0.864	0.874	0.114	0.837	6.52
0.7	0.880	0.969	0.028	0.833	9.03
1.0	0.883	0.948	0.046	0.837	12.73

관찰: - L2 변형량은 s에 거의 선형 비례 (가운데 그림). 워핑 강도와 시각적 perturbation은 1:1 관계 - **ASR**은 비선형 S자 곡선 — s=0.1에서 7%(거의 학습 안 됨) → s=0.3에서 54% → s=0.5~0.7 사이 87%~97%로 가파른 상승 → s=0.7에서 피크 → s=1.0에서 미세 saturate (97%→95%) - **clean ACC와 cross ACC는 모든 s에서 0.83~0.88 사이 평탄** — 워핑 강도와 무관하게 정상 일반화 유지 - **Pareto frontier** (오른쪽 그림)에서 **s=0.5가 명확한 knee** — 그 이후로 perturbation은 ×2로 늘지만 ASR 증가폭은 ≤10%p - 30 epoch만으로도 s≥0.5 영역에서 본 보고서 메인 결과(100 epoch에서 ASR 0.95)와 거의 같은 수준 도달 → WaNet 백도어가 빠르게 수렴

결론: 원 논문의 기본값 s=0.5 는 (인지 불가능성, 공격 성공률) 두 축의 균형점이라는 주장을 정량적으로 재확인. s=0.7 은 ASR을 살짝 더 올릴 수 있지만 L2가 38% 증가 — 비용 대비 효과 낮음. s=0.1 은 학습이 거의 안 되며, s≥1.0 은 명확히 stealth가 깨진다.

6. 평가 지표 사전

지표	풀네임	정의	좋은 값
ACC (test_acc)	Clean Accuracy	clean 입력의 정상 라벨 분류율	공격: 높을수록 stealth / 방어: 높을수록 좋음
ASR (test_asr)	Attack Success Rate	트리거 입력이 target 클래스로 분류된 비율	공격: ↑ / 방어: ↓
RA (test_ra)	Robust/Recovery Accuracy	트리거 입력이 원래 라벨로 분류된 비율	방어: ↑
Cross ACC	Cross-trigger Accuracy	(WaNet 전용) noise grid만 적용한 입력의 정상 분류율	높아야 정상 (단순 변형 ≠ 트리거 검증)

관계: $ASR + RA \leq 1$. 공격 강력 = ASR↑ & ACC 유지, 방어 효과 = ASR↓ & ACC 유지 & RA↑.

부가 지표 (csv에 기록): - train_* : 학습셋 분할별 (clean / bd / cross) ACC·loss -

bd_test_loss_avg_over_batch, cross_test_loss_avg_over_batch, ra_test_loss_avg_over_batch : 각 평가 split 평균 loss

7. 산출물 구성

종류	내용
학습된 백도어 모델 ckpt	공격 후 weights + 메타 정보 (방어 평가의 입력)
epoch별 메트릭 csv	train/test의 ACC·ASR·RA·Cross-loss
학습 곡선 PNG	ACC/ASR/RA/Cross, loss
변형 데이터셋 PNG	clean / bd / cross — 인텍스 매칭 비교용
방어 후 모델 ckpt	FT, NAD, ANP, NC 각각
방어별 메트릭 csv	clean ACC, ASR, RA
분석 시각화 PNG	t-SNE, Grad-CAM, frequency, confusion matrix (공격/방어 두 버전)
NC reverse trigger	클래스별 reverse-engineered L1 norm 텍스트 출력

8. 결론

1. **WaNet은 stealthy 공격으로 강함** — CIFAR-10에서 ACC 91.4% 유지, ASR 95.1%. clean과 백도어 이미지는 육안 구별 불가.
2. **마지막 conv layer의 attention은 보존된다** — Grad-CAM에서 공격 모델은 clean과 동일한 영역(객체)을 보고도 잘못된 라벨을 출력. NAD 후엔 attention 거의 동일한데 분류만 회복. 결정 경계가 바뀐 것은 분명하나, "어느 layer가 백도어를 보유하는가"는 attention만으로 알 수 없음.
3. **백도어는 분류 헤드가 아니라 중-후반 conv stage + BN bias에 분산** — §5.6의 layer-wise weight delta에서 NAD가 가장 적게 수정한 stage가 `linear` (분류 헤드, 0.029), 가장 많이 수정한 stage가 `layer3` (0.049). 단일 파라미터 1위는 `layer4.1.bn2.bias`. BN affine 파라미터가 Top-20의 다수를 차지 → BN과 후반 conv block이 백도어 결정 경로의 핵심 노드.
4. **ANP는 작은 mask로 정화 가능** — §5.5 threshold sweep에서 $t \in [0, 0.65]$ 구간은 ACC 무손실 정화 영역. Pareto knee $t=0.45$ 는 ACC 0.9064 / ASR 0.0114 / RA 0.8952 — §4.1의 ANP 단일 실행값보다 더 나은 trade-off. WaNet 백도어는 소수 뉴런 마스킹만으로도 깨진다는 정량 증거.
5. **ABL은 epoch 예산 민감** — 축소 셋팅(40 epoch)에서 ASR 0.59로 부분 정화에 그쳤다. FT/NAD/ANP는 비슷한 epoch에서 ASR을 0.1 이하로 떨어뜨린 반면, ABL의 isolation+unlearning 파이프라인은 unlearning stage가 충분히 진행돼야 효과가 나오는 구조 → 같은 epoch budget에서 다른 방어 대비 비효율적임을 보여주는 negative 데이터 포인트.
6. **NC는 무력화됨** — 모든 클래스의 reverse trigger L1 norm이 90~140으로 유사 → MAD anomaly 미검출. NC가 가정하는 "작은 고정 패턴" 가설을 위핑이 위반한다는 원 논문의 주장을 실험적으로 재확인.
7. **Cross 샘플 메커니즘이 stealth의 핵심** — cross 샘플은 시각적으로 bd보다 더 변형이 크다(§3.5의 3행). 모델이 "임의 변형이 아닌 특정 위핑"만 트리거로 학습하게 강제하는 디코이.
8. **위핑 강도 $s=0.5$ 는 Pareto knee로 정량 검증됨** — §5.7 ablation에서 L2 perturbation은 s 에 선형, ASR은 비선형 S자 곡선. $s=0.5$ 에서 ASR 0.87, $s=0.7$ 에서 0.97 (피크) — $s=0.5$ 이후 ASR 증가 폭은 둔화되고 perturbation은 빠르게 증가. 원 논문의 기본값 선택이 합리적임을 실험적으로 확인.

부록 A: 라이브러리 호환 패치

NumPy 2.0 / PyTorch 2.6+ / grad-cam 1.5+ 의 API 변화에 따라 다음 4건 수정:

- `np.infty` → `np.inf` (NumPy 2.0)
- `torch.load(...)` 에 `weights_only=False` 명시 2곳 (PyTorch 2.6 default 변화)
- `FullGrad(...)` 호출에서 deprecate된 `use_cuda` 인자 제거 (grad-cam 1.5+)

분석 시각화는 (1) clean/bd/cross 3행 비교와 (2) 방어 비교 bar chart 두 가지를 별도 파이썬 스크립트로 작성해 사용했다.